

Universidade Federal do Ceará - Campus Quixadá

Disciplina: Sistemas Distribuídos

Professor: Rafael Braga

Alunos: Erica de Castro Silveira, Francisco Leudes Bezerra Neto

Relatório de Implementação de Serviço Remoto via API: WPlace (Quadro Colaborativo).

1) Introdução

O objetivo deste trabalho foi reimplementar o serviço remoto desenvolvido anteriormente, migrando da comunicação de baixo nível (Sockets) ou RMI para um modelo baseado em Web Services (WS) e API REST.

O sistema desenvolvido, inspirado na plataforma "WPlace", simula um quadro de pintura colaborativa em tempo real. Ele segue a arquitetura cliente-servidor, utilizando o protocolo HTTP para requisição e resposta. O servidor centraliza o estado do quadro (quais pixels estão pintados e suas cores), enquanto múltiplos clientes heterogêneos podem visualizar e modificar esse quadro simultaneamente.

2) Funcionalidade do Sistema

O Sistema consiste em uma grade (matriz) de 30x30 pixels. As principais funcionalidades são:

1. **Visualização de Estado:** Os clientes consultam periodicamente o servidor para atualizar a visualização do quadro localmente, permitindo ver o que outros usuários desenharam.
2. **Pintura de Pixels:** O usuário seleciona uma cor e clica (ou arrasta o mouse) sobre a grade. O cliente envia as coordenadas (X, Y), a cor (RGB) e o nome do usuário para o servidor.
3. **Persistência em Memória:** O servidor mantém um dicionário de dados contendo o estado atual de cada pixel pintado.
4. **Limpeza (Admin):** Uma funcionalidade administrativa para limpar todo o quadro instantaneamente.

3) Tecnologias Utilizadas

Para atender aos requisitos de heterogeneidade e interoperabilidade, utilizamos tecnologias distintas para o servidor e para os dois tipos de clientes:

- **Servidor (Back-end):**
 - **Linguagem:** Python 3.
 - **Framework:** Utilizamos o **FastAPI**. Escolhido pela sua rapidez, validação automática de dados (via Pydantic) e facilidade na criação de APIs RESTful. O servidor roda sobre o servidor ASGI *Uvicorn*.

- **Clientes (Front-end):**

- **Cliente 1 (Web):** Desenvolvido em **HTML5 e JavaScript**. Utiliza a *Canvas API* para renderização gráfica e a *Fetch API* nativa do navegador para comunicação assíncrona com o servidor.
- **Cliente 2 (Desktop):** Desenvolvido em **C# (.NET 8.0)** utilizando **Windows Forms**. A comunicação HTTP é realizada através da classe `HttpClient` e a serialização de dados utiliza `System.Net.Http.Json`.

4) Descrição da API

O servidor expõe 3 *endpoints* (rotas) principais que manipulam o recurso "Board" . A comunicação utiliza o formato **JSON** para troca de dados.

Método HTTP	Endpoint	Descrição da Funcionalidade	Parâmetros (Entrada JSON)	Resposta (Saída)
GET	<code>/board</code>	Retorna o estado atual de todos os pixels pintados no quadro.	<i>Nenhum</i>	JSON contendo um dicionário onde a chave é "x,y" e o valor é o objeto do pixel.
POST	<code>/paint</code>	Pinta um pixel específico no servidor.	<code>{ "x": int, "y": int, "r": int, "g": int, "b": int, "owner": str }</code>	<code>{ "status": "painted", "pixel": {...} }</code> ou Erro 400 se coordenadas inválidas.
DELETE	<code>/clear</code>	Limpa todos os pixels do quadro (reseta o estado).	<i>Nenhum</i>	<code>{ "status": "board cleared" }</code>

5) Detalhes de Implementação e Interação

Servidor: O servidor mantém uma instância da classe `Board`, que possui um dicionário `self.pixels`. A escolha de um dicionário (Hash Map) em vez de uma matriz 2D simples foi feita para otimizar o tráfego de rede: apenas os pixels que foram efetivamente pintados são armazenados e enviados no JSON, economizando banda se o quadro estiver vazio ou parcialmente preenchido. O servidor também implementa *CORS* (*Cross-Origin Resource Sharing*) para permitir que o cliente Web (JavaScript) faça requisições de uma origem diferente.

Clientes e Concorrência: Como o protocolo HTTP é *stateless* (sem estado de conexão persistente como sockets TCP puros), a atualização em tempo real foi simulada através de **Polling**.

- Tanto o cliente **JavaScript** (via `setInterval`) quanto o cliente **C#** (via `System.Windows.Forms.Timer`) consultam a rota `GET /board` a cada 1 segundo.
- Ao receber os dados, ambos os clientes redesenham a grade.
- Para garantir fluidez na pintura, ambos os clientes implementam "Feedback Visual Otimista": eles pintam o pixel na tela localmente no momento do clique e, em seguida, enviam a requisição `POST /paint` para o servidor validar e propagar para os outros usuários.

6) Conclusão

A implementação deste trabalho demonstrou a eficácia do uso de APIs REST para a construção de sistemas distribuídos heterogêneos. A separação clara entre a regra de negócio (API Python) e a interface (Web e Desktop) permitiu que desenvolvêssemos clientes em linguagens completamente diferentes (Javascript e C#) consumindo o mesmo serviço sem necessidade de adaptação no servidor. Essa abordagem simplificou drasticamente o tratamento de conexões e formatos de dados em comparação ao uso de Sockets puros do trabalho anterior.

